

# halfedge：示例集设计文档

examples/

halfedge 库

2026-07-04

## 目录

|          |                                  |          |
|----------|----------------------------------|----------|
| <b>1</b> | <b>示例总览</b>                      | <b>2</b> |
| <b>2</b> | <b>示例依赖关系</b>                    | <b>3</b> |
| <b>3</b> | <b>各示例说明</b>                     | <b>3</b> |
| 3.1      | storage_basic: 底层存储 CRUD         | 3        |
| 3.2      | build_mesh: 从顶点面索引构建网格           | 3        |
| 3.3      | obj_io: OBJ 读写                   | 3        |
| 3.4      | traversal: 邻域遍历                  | 3        |
| 3.5      | topology_ops: 三大拓扑操作             | 4        |
| 3.6      | extrude_face: 三角面挤出              | 4        |
| 3.7      | geometry_query: 几何查询             | 4        |
| 3.8      | laplacian_smooth: 拉普拉斯平滑         | 4        |
| 3.9      | point_triangle_distance: 点到三角形距离 | 5        |
| 3.10     | loop_subdivision: Loop 细分        | 5        |
| 3.11     | property: OpenMesh 风格属性系统        | 5        |
| 3.12     | validate: 拓扑自检                   | 6        |
| 3.13     | export_wgpu: wgpu 缓冲导出           | 6        |
| 3.14     | icosphere: 球体生成与性质验证             | 6        |
| <b>4</b> | <b>运行方式</b>                      | <b>6</b> |
| 4.1      | 运行单个示例                           | 6        |
| 4.2      | 编译所有示例（不运行）                      | 6        |
| 4.3      | 列出所有可用示例                         | 7        |
| <b>5</b> | <b>示例设计原则</b>                    | <b>7</b> |
| <b>6</b> | <b>典型输出示例</b>                    | <b>7</b> |
| 6.1      | icosphere 输出片段                   | 7        |
| 6.2      | topology_ops 输出片段                | 7        |
| 6.3      | loop_subdivision 输出片段            | 8        |

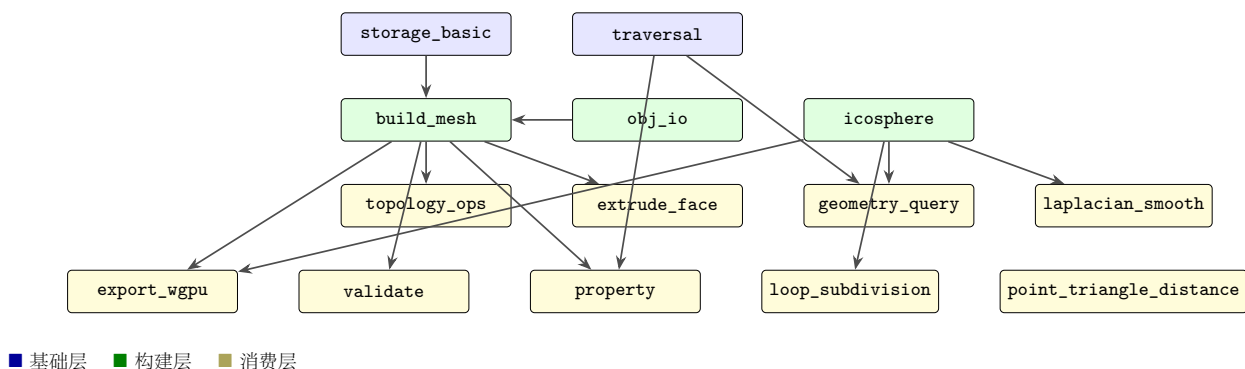
|     |                             |   |
|-----|-----------------------------|---|
| 6.4 | extrude_face 输出片段 . . . . . | 8 |
| 6.5 | property 输出片段 . . . . .     | 8 |

## 1 示例总览

examples/ 目录下提供 14 个独立可运行的示例，按功能划分，每个示例聚焦一个模块的核心用法。所有示例通过 `cargo run --example <name>` 运行，无需额外依赖。

| 示例                      | 模块           | 功能                       | 运行命令                                                     |
|-------------------------|--------------|--------------------------|----------------------------------------------------------|
| storage_basic           | storage      | 顶点/半边/面 CRUD 与句柄有效性      | <code>cargo run --example storage_basic</code>           |
| build_mesh              | io           | 从顶点 + 面索引构建完整网格          | <code>cargo run --example build_mesh</code>              |
| obj_io                  | io           | OBJ 文本/文件读写往返            | <code>cargo run --example obj_io</code>                  |
| traversal               | traversal    | 5 迭代器 + 2 边界判定函数         | <code>cargo run --example traversal</code>               |
| topology_ops            | topology_ops | split/flip/collapse 三大操作 | <code>cargo run --example topology_ops</code>            |
| extrude_face            | topology_ops | 单/批量三角面挤出形成棱柱            | <code>cargo run --example extrude_face</code>            |
| geometry_query          | geometry     | 边长/面积/法向/最小内角/顶点法向       | <code>cargo run --example geometry_query</code>          |
| laplacian_smooth        | geometry     | 拉普拉斯平滑（顶点级 + 整网级）        | <code>cargo run --example laplacian_smooth</code>        |
| point_triangle_distance | geometry     | 点到三角形 7-Voronoi 区域距离     | <code>cargo run --example point_triangle_distance</code> |
| loop_subdivision        | subdiv       | Loop 细分规模/度数/Euler 验证    | <code>cargo run --example loop_subdivision</code>        |
| property                | property     | OpenMesh 风格属性系统全演示       | <code>cargo run --example property</code>                |
| validate                | validate     | 拓扑自检：干净通过 + 破坏检测         | <code>cargo run --example validate</code>                |
| export_wgpu             | export       | 导出 wgpu 顶点/索引缓冲          | <code>cargo run --example export_wgpu</code>             |
| icosphere               | test_util    | icosphere 生成与性质验证        | <code>cargo run --example icosphere</code>               |

## 2 示例依赖关系



## 3 各示例说明

### 3.1 storage\_basic: 底层存储 CRUD

演示 MeshStorage 的增删改查与 slotmap 的 ABA 安全保证。关键点:

- 创建 3 顶点 + 3 半边 + 1 面, 手动设置 next/prev/face 环;
- 删除顶点后句柄立即失效, contains\_vertex 返回 false;
- 槽位复用后旧句柄不复活 (版本号机制)。

### 3.2 build\_mesh: 从顶点面索引构建网格

演示 build\_mesh\_from\_vertices\_faces, 自动构建 twin/next/prev/边界环。关键点:

- 输入 4 顶点 + 2 三角形 (共享 1 边), 输出 10 半边 (2 内部 + 8 边界);
- 用 validate\_topology 验证构建结果;
- 用 is\_boundary\_edge 区分内部边与边界边。

### 3.3 obj\_io: OBJ 读写

演示 parse\_obj / format\_obj / save\_obj / load\_obj。关键点:

- 解析四面体 OBJ 文本, 要求面绕向一致;
- 序列化回 OBJ 文本, 文件 IO 往返一致;
- 支持 v/vt/vn 格式与负索引。

### 3.4 traversal: 邻域遍历

演示 5 个迭代器 + 2 个边界判定函数。关键点:

- VertexRing: 顶点的 outgoing 半边环 (CCW 顺序);

- `VertexAdjacentVerts / VertexAdjacentFaces`: 邻居顶点 / 邻接面;
- `FaceHalfEdges / FaceVertices`: 面边界环;
- 迭代期不持有借用, 可自由 `&mut mesh`。

### 3.5 topology\_ops: 三大拓扑操作

演示 `split_edge / flip_edge / collapse_edge`。关键点:

- `split`: 顶点 +1, 面 +2 (内部边) 或 +1 (边界边);
- `flip`: 顶点/面/半边数不变, 仅重连 4 条半边;
- `collapse`: 顶点 -1, 面 -2, 半边 -6;
- 翻转边界边返回 `FlipOnBoundaryEdge` 错误。

### 3.6 extrude\_face: 三角面挤出

演示 `extrude_face / extrude_faces` 将三角面沿给定方向 `offset` 挤出形成闭合三棱柱。关键点:

- 单面挤出: 新增 3 顶点 + 7 面 (1 反向顶面 + 6 侧面三角形),  $Euler = 2$ ;
- 不同 `offset` 方向 (沿法向、斜向、反向) 都能正确闭合;
- 零向量 `offset` 返回 `DegenerateTriangle`;
- 与三角形共面的 `offset` (侧面面积  $< 10^{-12}$ ) 同样报退化;
- 批量挤出 `extrude_faces` 对不相交三角形集合一次性挤出;
- 挤出后所有面边界环长度均为 3, `validate_topology` 通过。

### 3.7 geometry\_query: 几何查询

演示 5 个查询函数: 边长、面积、法向、最小内角、顶点法向。关键点:

- `icosphere(0)` 所有面为等边三角形, 最小内角 =  $60^\circ$ ;
- 总面积  $\approx 4\pi$  (单位球面面积);
- 顶点法向与顶点位置平行 (球面性质)。

### 3.8 laplacian\_smooth: 拉普拉斯平滑

演示 `laplacian_smooth_vertex` 与 `laplacian_smooth_mesh`。关键点:

- 单顶点: 新位置 = 邻居位置算术平均;
- 整网:  $\lambda = 0.5$ , 5 次迭代, 重心偏移  $< 10^{-15}$  (保留重心);
- 平滑使球面收缩 (顶点偏离单位球面);
- $\lambda = 0$  时网格不变。

### 3.9 point\_triangle\_distance: 点到三角形距离

演示 `Ericson 7-Voronoi` 区域算法。关键点:

- 面域: 投影在三角形内部, 距离 = 垂直高度;
- 顶点域  $V_A/V_B/V_C$ : 距离到对应顶点;
- 边域  $E_{AB}/E_{AC}/E_{BC}$ : 距离到边投影;
- 退化三角形 (共线) 退化为三顶点最近距离。

### 3.10 loop\_subdivision: Loop 细分

演示 `subdiv::loop_subdivide` 在 `icosphere` 上的光滑细分效果。关键点:

- 单次细分规模:  $V' = V + E$ ,  $F' = 4F$  (`icosphere(1)`  $42 \rightarrow 162$  顶点,  $80 \rightarrow 320$  面);
- 连续多次细分保持 `check_topology` 通过;
- 度数分布: 原始 12 顶点保持度数 = 5, 新增边中点度数 = 6 (正则性质);
- Euler 示性数  $V - E + F = 2$  在每次细分后保持;
- Loop 细分是逼近型细分, 单位球面偏差随细分次数减小;
- `validate_topology` 完整校验无错误。

### 3.11 property: OpenMesh 风格属性系统

演示 `property::MeshProperties` 与 `PropertyHandle<T>` 的核心用法。关键点:

- 注册 f64 顶点权重属性, `set/get_vertex_prop` 读写;
- 类型擦除: 同一容器并存 f64 / i32 / String / bool 多种属性;
- 加权拉普拉斯平滑: 北极顶点权重 5.0, 其余 1.0, 验证等权与加权位移;
- 半边属性: 缓存所有半边边长, 统计 min/max/avg;
- 面属性: 缓存面法向, 与面心方向一致;
- `remove_vertex_with_props` 删除顶点时同步清理属性;
- newtype 模式: 同底层 f64 通过 `Weight/Temperature` 注册两个互不冲突的属性。

### 3.12 validate: 拓扑自检

演示 `validate_topology` 的完整校验。关键点:

- 干净 `icosphere(1)` 通过校验 (0 错误);
- 人为破坏 `twin` 互指 → 检测 `TwinMismatch + SelfLoopHalfEdge`;
- 悬空顶点引用 → `HalfEdgeDanglingVertex`;
- 共线三角形 → `DegenerateFace`;
- 入口 `origin` 不匹配 → `VertexHalfEdgeInconsistent`。

### 3.13 export\_wgpu: wgpu 缓冲导出

演示 `mesh_to_vertex_index_buffers`。关键点:

- 输出 `Vec<f32; 3>` 顶点 + `Vec<u32>` 索引;
- `icosphere(1)`: 42 顶点 (504 字节) + 240 索引 (960 字节);
- 索引范围合法 (`max index < vertex count`);
- CCW 朝向: 法向 · 重心  $> 0$  (朝外)。

### 3.14 icosphere: 球体生成与性质验证

演示 `build_icosphere` 在不同细分级别的网格性质。关键点:

- 规模公式:  $V_n = 10 \cdot 4^n + 2$ ,  $F_n = 20 \cdot 4^n$ ,  $E_n = 30 \cdot 4^n$ ;
- 所有顶点在单位球面上 (偏差  $< 10^{-15}$ );
- 所有面法向朝外;
- `icosahedron(0)` 的 12 个顶点度数均为 5;
- Euler 公式  $V - E + F = 2$  成立。

## 4 运行方式

### 4.1 运行单个示例

```
1 cargo run --example storage_basic
2 cargo run --example icosphere
```

### 4.2 编译所有示例 (不运行)

```
1 cargo build --examples
```

### 4.3 列出所有可用示例

```
1 cargo run --example # Tab
```

## 5 示例设计原则

1. 自包含：每个示例独立可运行，不依赖其他示例；
2. 聚焦单一功能：每个示例只演示一个模块的核心 API；
3. 可观测输出：所有示例用 `println!` 打印关键数据，便于直观验证；
4. 使用真实网格：大多数示例以 `build_icosphere` 作为测试网格，保证拓扑合法、几何有意义；
5. 包含错误处理：`topology_ops` 与 `validate` 示例演示了错误路径（翻转边界边、检测破坏）。

## 6 典型输出示例

### 6.1 icosphere 输出片段

```
1 === icosphere          ===
2
3 [
4     n
5     0      12      20      60      OK
6     1      42      80     240      OK
7     2     162     320     960      OK
8     3     642    1280    3840      OK
9     4    2562    5120   15360      OK
10
11 [  1:      ] icosphere(2)          = 2.22e-16
12 [  5: Euler] n=0: V-E+F = 12-30+20 = 2      2
```

### 6.2 topology\_ops 输出片段

```
1 [split_edge]          VertexId(13v1)
2     12 → 13
3     20 → 22
4     validate_mesh: Ok(())
5     check_topology: OK
6
7 [collapse_edge]        VertexId(13v1)
8     12 → 11
9     20 → 18
10    60 → 54
11
12          → Some("HalfEdgeId(1v1)")
```

### 6.3 loop\_subdivision 输出片段

```
1 [1]      Loop
2      icosphere(1): V=   42 F=   80 E=  120 HE=  240
3              : V=  162 F=  320 E=  480 HE=  960
4      V'=V+E (42+120=162) F'=4F (4*80=320)
5      OK
6
7 [4] Euler      V - E + F
8      0x: V-E+F = 12-30+20 = 2      2
9      1x: V-E+F = 42-120+80 = 2      2
10     2x: V-E+F = 162-480+320 = 2      2
11     3x: V-E+F = 642-1920+1280 = 2      2
```

### 6.4 extrude\_face 输出片段

```
1 [1]
2      : V=   3 F=   1 E=   3 HE=   6
3      : V=   6 F=   8 E=  12 HE=  24
4      = 7      7 1      + 6
5      OK
6 Euler      V-E+F = 2      2
7
8 [3]      /      offset
9      offset DegenerateTriangle
10 x      offset DegenerateTriangle
```

### 6.5 property 输出片段

```
1 ===== 1.      =====
2      f64      42
3 vertex_prop_type_count = 1
4
5 ===== 2.      =====
6      i32 / String / bool
7 vertex_prop_type_count = 4
8
9 ===== 4.      =====
10     240      halfedge_prop_type_count = 1
11     min=0.6071, max=0.7174, avg=0.6760 ( 240      )
12
13 ===== 6.      =====
14
15 vVertexId(1v1) exists=true, weight=Some(1.0)
16
17 vVertexId(1v1) exists=false, weight=None
```